

Semana 9 – Sesión 2 (18)

Repaso de NumPy y Matplotlib

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Índice

Objetivo de la sesión

Recordatorio de NumPy

Recordatorio de Matplotlib

Ejercicios guiados

Ejercicios tipo Solemne

Errores comunes

Cierre

Objetivo de la sesión

Objetivo de la sesión

Objetivo de la sesión

- Repasar la creación de arreglos con **NumPy**.
- Practicar operaciones vectorizadas y funciones matemáticas.
- Reforzar gráficos básicos con **Matplotlib**.
- Resolver ejercicios similares al razonamiento esperado en la Solemne II.

Recordatorio de NumPy

Recordatorio de NumPy

Crear arreglos

Herramientas mínimas para hoy.

```
1 import numpy as np
2
3 a = np.array([1, 2, 3, 4])
4 x = np.linspace(0, 10, 100)
5 n = np.arange(0, 20, 2)
```

- `np.array`: transforma datos en arreglo.
- `np.linspace`: puntos igualmente espaciados.
- `np.arange`: secuencia con paso fijo.

Recordatorio de NumPy

Operaciones vectorizadas

Idea clave. Operar sobre todo el arreglo sin escribir un `for`.

```
1 t = np.linspace(0, 5, 50)
2 x0 = 2.0
3 v0 = 3.0
4 a  = -1.5
5
6 x = x0 + v0*t + 0.5*a*t**2
7
8 print(x.mean(), x.min(), x.max())
```

Punto docente. Las operaciones se aplican elemento a elemento.

Recordatorio de Matplotlib

Recordatorio de Matplotlib

Gráfico de línea

Estructura básica de una figura.

```
1 import matplotlib.pyplot as plt
2
3 plt.plot(t, x, label="posicion")
4 plt.xlabel("tiempo [s]")
5 plt.ylabel("posicion [m]")
6 plt.title("Movimiento en 1D")
7 plt.grid(True)
8 plt.legend()
9 plt.show()
```

Dos gráficos frecuentes en ejercicios.

```
1 plt.scatter(t, x, alpha=0.7)
2 plt.xlabel("tiempo [s]")
3 plt.ylabel("posicion [m]")
4 plt.grid(True)
5 plt.show()
6
7 plt.hist(x, bins=12, edgecolor="black")
8 plt.xlabel("posicion [m]")
9 plt.ylabel("frecuencia")
10 plt.show()
```

Ejercicios guiados

Enunciado

- Crea un arreglo `t` con 100 valores entre 0 y 8 segundos.
- Usa $x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$, con $x_0 = 0$, $v_0 = 12$ y $a = -9.8$.
- Grafica posición versus tiempo.
- Calcula la altura máxima usando NumPy.

Conceptos: `linspace`, operaciones vectorizadas, `max`, `plot`.

Ejercicios guiados

Solución Guiado 1:

Movimiento 1D



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 t = np.linspace(0, 8, 100)
4 x0 = 0
5 v0 = 12
6 a = -9.8
7 x = x0 + v0*t + 0.5*a*t**2
8 altura_maxima = np.max(x)
9 print("Altura maxima:", altura_maxima)
10 plt.plot(t, x)
11 plt.xlabel("tiempo [s]")
12 plt.ylabel("posicion [m]")
13 plt.title("Movimiento 1D")
14 plt.grid(True)
15 plt.show()
```

Discusión: El arreglo `t` contiene todos los tiempos y la formula se aplica a todos sus valores al mismo tiempo.

Enunciado

- Crea un arreglo `theta` entre 0 y 2π .
- Define $r = 2 + \cos(\theta)$.
- Calcula $x = r \cos(\theta)$ e $y = r \sin(\theta)$.
- Grafica la trayectoria y usa `plt.axis("equal")`.

Conceptos: coordenadas polares, `np.cos`, `np.sin`, ejes proporcionales.

Ejercicios guiados

Solución Guiado 2:

Trayectoria 2D



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 theta = np.linspace(0, 2*np.pi, 200)
4 r = 2 + np.cos(theta)
5 x = r*np.cos(theta)
6 y = r*np.sin(theta)
7 plt.plot(x, y)
8 plt.xlabel("x")
9 plt.ylabel("y")
10 plt.title("Trayectoria 2D")
11 plt.axis("equal")
12 plt.grid(True)
13 plt.show()
```

Discusión: Primero se calcula el radio y luego se transforma cada punto desde coordenadas polares a coordenadas cartesianas.

Enunciado

- Genera 200 mediciones alrededor de 5.0 usando `np.random.normal`.
- Calcula media y desviación estándar.
- Dibuja un histograma de las mediciones.
- Marca la media con una línea vertical.

Conceptos: números aleatorios, estadística descriptiva, histogramas.

Ejercicios guiados

Solución Guiado 3:

Datos con ruido



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(1)
4 mediciones = np.random.normal(5.0, 0.3, 200)
5 media = np.mean(mediciones)
6 desviacion = np.std(mediciones)
7 print("Media:", media)
8 print("Desviacion estandar:", desviacion)
9 plt.hist(mediciones, bins=15, edgecolor="black")
10 plt.axvline(media, color="red", label="media")
11 plt.xlabel("medicacion")
12 plt.ylabel("frecuencia")
13 plt.title("Medicaciones con ruido")
14 plt.legend()
15 plt.show()
```

Discusión: La media indica el valor central de las mediciones y la desviación estándar indica cuánto se dispersan.

Enunciado

- Crea arreglos para distancia d y velocidad v .
- Calcula $H = v/d$ para cada dato.
- Grafica v versus d usando **scatter**.
- Calcula el promedio de H .

Conceptos: división elemento a elemento, **scatter**, promedio.

Ejercicios guiados

Solución Guiado 4:

Relación física



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 d = np.array([2, 4, 6, 8, 10])
4 v = np.array([140, 260, 430, 540, 700])
5 H = v/d
6 promedio_H = np.mean(H)
7 print("Valores de H:", H)
8 print("Promedio de H:", promedio_H)
9 plt.scatter(d, v)
10 plt.xlabel("distancia")
11 plt.ylabel("velocidad")
12 plt.title("Velocidad versus distancia")
13 plt.grid(True)
14 plt.show()
```

Discusión: Al dividir dos arreglos del mismo tamaño, NumPy divide cada velocidad por su distancia correspondiente.

Ejercicios tipo Solemne



Enunciado

- Construye t entre 0 y 4 segundos con 80 puntos.
- Calcula $y(t) = 1.5 + 18t - 4.9t^2$.
- Grafica y versus t , con etiquetas y grilla.
- Informa la altura máxima y el tiempo donde ocurre.

Conceptos: arreglo temporal, vectorización, `np.max`, `np.argmax`.



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 t = np.linspace(0, 4, 80)
4 y = 1.5 + 18*t - 4.9*t**2
5 altura_maxima = np.max(y)
6 indice_maximo = np.argmax(y)
7 tiempo_maximo = t[indice_maximo]
8 print("Altura maxima:", altura_maxima)
9 print("Tiempo de altura maxima:", tiempo_maximo)
10 plt.plot(t, y)
11 plt.xlabel("tiempo [s]")
12 plt.ylabel("altura [m]")
13 plt.title("Altura de un proyectil")
14 plt.grid(True)
15 plt.show()
```

Discusión: `np.argmax` entrega la posición del valor máximo; con esa posición se obtiene el tiempo correspondiente.



Enunciado

- Crea θ entre 0 y 2π con 360 puntos.
- Define $r = 1 + 0.15 \sin(5\theta)$.
- Calcula $x = r \cos(\theta)$ e $y = r \sin(\theta)$.
- Grafica la órbita usando `axis("equal")`.

Conceptos: trigonometría con NumPy, trayectoria 2D, proporción de ejes.

Solución 2 de Referencia: Órbita circular perturbada



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 theta = np.linspace(0, 2*np.pi, 360)
4 r = 1 + 0.15*np.sin(5*theta)
5 x = r*np.cos(theta)
6 y = r*np.sin(theta)
7 plt.plot(x, y)
8 plt.xlabel("x")
9 plt.ylabel("y")
10 plt.title("Órbita circular perturbada")
11 plt.axis("equal")
12 plt.grid(True)
13 plt.show()
```

Discusión: La parte $0.15 \cdot \sin(5 \cdot \theta)$ modifica suavemente el radio y produce la perturbación de la órbita.



Enunciado

- Simula 300 mediciones de una longitud promedio $L = 2.4$ m.
- Usa una dispersión de 0.08 m.
- Calcula media, desviación estándar, mínimo y máximo.
- Dibuja un histograma e interpreta la dispersión.

Conceptos: `np.random.normal`, `mean`, `std`, `hist`.

Ejercicios tipo Solemne

Solución 3 de Referencia:

Mediciones experimentales



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(2)
4 mediciones = np.random.normal(2.4, 0.08, 300)
5 media = np.mean(mediciones)
6 desviacion = np.std(mediciones)
7 minimo = np.min(mediciones)
8 maximo = np.max(mediciones)
9 print("Media:", media)
10 print("Desviacion estandar:", desviacion)
11 print("Minimo:", minimo)
12 print("Maximo:", maximo)
13 plt.hist(mediciones, bins=15, edgecolor="black")
14 plt.xlabel("longitud [m]")
15 plt.ylabel("frecuencia")
16 plt.title("Mediciones de longitud")
17 plt.show()
```

Discusión: La dispersión de 0.08 m hace que la mayoría de las mediciones quede cerca de 2.4 m.



Enunciado

- Crea arreglos de distancias [5, 10, 20, 40, 80] Mpc.
- Crea velocidades [350, 710, 1420, 2790, 5650] km/s.
- Grafica velocidad versus distancia.
- Calcula $H_0 = v/d$ y estima su promedio.

Conceptos: arrays, scatter plot, división vectorizada, promedio.

Solución 4 de Referencia: *Ley de Hubble simplificada*



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 d = np.array([5, 10, 20, 40, 80])
4 v = np.array([350, 710, 1420, 2790, 5650])
5 H0 = v/d
6 promedio_H0 = np.mean(H0)
7 print("Valores de H0:", H0)
8 print("Promedio de H0:", promedio_H0)
9 plt.scatter(d, v)
10 plt.xlabel("distancia [Mpc]")
11 plt.ylabel("velocidad [km/s]")
12 plt.title("Ley de Hubble simplificada")
13 plt.grid(True)
14 plt.show()
```

Discusión: Si la relación es aproximadamente lineal, los valores v/d deberían ser parecidos entre sí.

Errores comunes

- Usar `math.sin` o `math.cos` sobre arreglos en vez de `np.sin` o `np.cos`.
- Olvidar `plt.show()` al final del gráfico.
- No usar `plt.axis("equal")` en trayectorias 2D.
- Mezclar listas y arrays sin revisar el tipo de dato.
- Entregar gráficos sin título, ejes o unidades.

Cierre

- NumPy permite construir arreglos y operar sobre ellos directamente.
- Matplotlib permite visualizar resultados con pocas instrucciones.
- La Solemne II exigirá combinar cálculo, gráfico e interpretación.

Cierre

Próxima sesión

La Semana 10, Sesión 1 queda como simulacro integrador para la Solemne II.

¡Fin de la sesión!